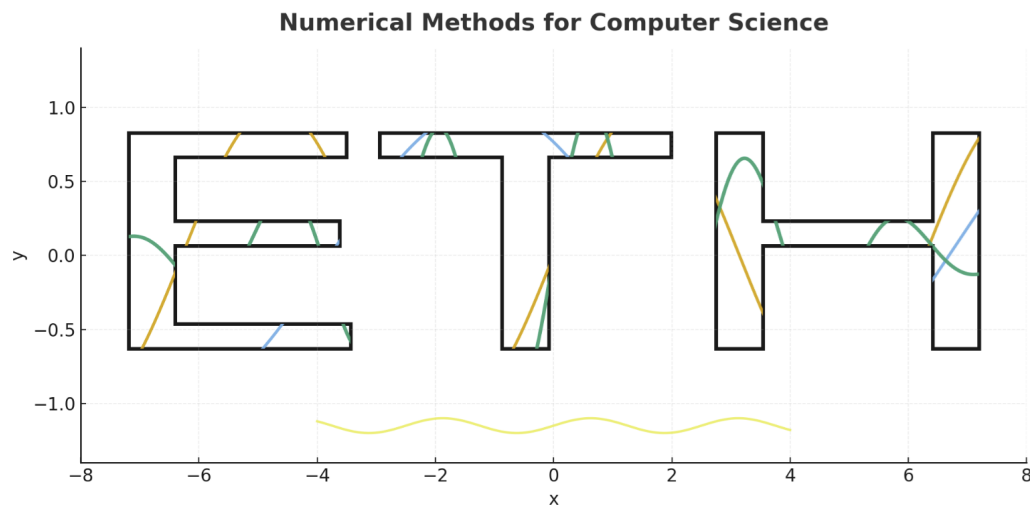




Applied Linear Algebra III — Week #12

ETH Zurich

DAVID Mihnea-Ştefan

mihdavid@ethz.ch

This week continues our study of applied linear algebra, focusing on advanced methods, structured matrices, and computational techniques relevant to modern numerical modeling.

Contents

1	Orthogonality in $\mathbb{R}^m$ and Orthogonal Matrices	1
1.1	Orthonormal Sets: n Vectors in $\mathbb{R}^m$	1
1.2	Consequences of $Q^\top Q = I_n$	1
1.3	Special Case: $m = n$ (Square Matrices)	1
1.4	Examples of Orthogonal Matrices	3
2	Projections with Orthonormal Bases	4
2.1	Projection onto a Subspace	4
2.1.1	Deriving the coefficients of the projection	4
3	QR Decomposition: Structure and Properties	5
3.1	Definition of the QR Decomposition	5
3.2	How QR is constructed	5
3.2.1	Defining the R matrix	5
3.2.2	Why $A = Q(Q^\top A)$	6
3.2.3	Why R is upper triangular	6
3.3	Least Squares via QR	7
4	Economical vs. Full QR Decomposition	9
4.1	Economical (thin) QR decomposition	9
4.2	Full QR decomposition	9
4.3	Comparison	10
5	Why Classical Gram–Schmidt Cannot Be Used in Practice	11
5.1	Geometric intuition for the instability	11
5.2	Consequences	12
5.3	What is used instead?	12
6	Computation of Orthogonal Transformations	13
6.1	A structural fact	13
6.2	Reflections	13
6.3	Why reflections are useful	13
6.3.1	Householder Reflections	14
6.4	Householder QR algorithm	16
6.5	Rotations (Givens Rotations)	17
7	Computational cost of Givens QR - Optional	22
8	Computational cost of Householder QR - Optional	24
9	Householder and Givens in Python	26
9.1	Householder QR in NumPy	26
9.2	Householder reflectors in SciPy	26
9.3	Givens rotations in SciPy	26

TA Award 2025

A note from your TA



Hello everyone! As you've probably already seen, VIS is once again organizing the **TA Award**, a prize given by students to the TA who resonated the most with them. If you feel that this might be me, and that I've managed, even a little, to make the NumCS course more digestible or interesting, I would be truly grateful if you considered voting for me.

On your side, it only takes a minute to open the link and cast your vote, but for me it would be a real confirmation that what I do actually has an impact. Knowing that my work has helped you, even in a small way, would mean a lot.

Thank you very much for your time and support!

1 Orthogonality in $\mathbb{R}^m$ and Orthogonal Matrices

1.1 Orthonormal Sets: n Vectors in $\mathbb{R}^m$

Let $q_1, q_2, \dots, q_n \in \mathbb{R}^m$ be n vectors (with $n \leq m$). We say that they form an **orthonormal set** if:

$$q_i^\top q_j = \delta_{ij}, \quad i, j = 1, \dots, n.$$

That is: - each vector has unit length: $\|q_i\| = 1$, - distinct vectors are perpendicular.

If we place them as columns of a matrix

$$Q = [q_1 \ q_2 \ \dots \ q_n] \in \mathbb{R}^{m \times n},$$

then the orthonormality relations can be written compactly as

$$Q^\top Q = I_n.$$

Note

This is the *most general* and *most useful* definition: Q has orthonormal columns but may be rectangular. It represents an n -dimensional orthonormal frame inside $\mathbb{R}^m$.

1.2 Consequences of $Q^\top Q = I_n$

From $Q^\top Q = I_n$ we deduce:

$$\text{rank}(Q) = n.$$

Thus the vectors $q_1, \dots, q_n$ are linearly independent, and Q maps $\mathbb{R}^n$ isometrically into $\mathbb{R}^m$.

For any $x \in \mathbb{R}^n$:

$$\|Qx\|^2 = x^\top (Q^\top Q)x = x^\top x = \|x\|^2.$$

Note

Rectangular $m \times n$ matrices with orthonormal columns arise in: QR decomposition, orthogonal projections, dimension reduction, and numerical optimization. Only the square case $m = n$ produces *orthogonal matrices*.

1.3 Special Case: $m = n$ (Square Matrices)

When $m = n$, the matrix $Q \in \mathbb{R}^{n \times n}$ is square, so we can discuss about the determinant.

Using the multiplicativity of the determinant:

$$\det(AB) = \det(A) \det(B),$$

apply it to the relation $Q^\top Q = I_n$:

$$1 = \det(I_n) = \det(Q^\top Q) = \det(Q^\top) \det(Q).$$

Since transposition preserves the determinant:

$$\det(Q^\top) = \det(Q),$$

we obtain:

$$(\det Q)^2 = 1.$$

Therefore:

$$\boxed{\det(Q) = \pm 1.}$$

Note

Geometric interpretation:

- $\det(Q) = +1 \implies Q$ is a **rotation**;
- $\det(Q) = -1 \implies Q$ is a **reflection**.

Orthogonal matrices preserve lengths, angles, volume (up to sign), and represent rigid motions.

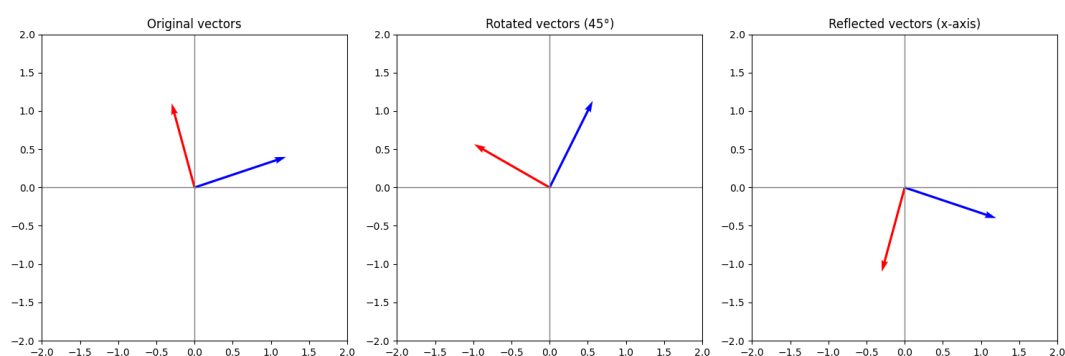


Figure 1: Rotation vs. reflection in $\mathbb{R}^2$.

The orthonormality condition becomes:

$$Q^\top Q = I_n.$$

This directly implies:

$$Q^\top = Q^{-1}.$$

This is because Q is invertible ($\det(Q) = \pm 1$), and the inverse must be unique. We multiply to the right with Q^{-1} .

Thus a **square matrix with orthonormal columns** is automatically invertible and is called an **orthogonal matrix**.

1.4 Examples of Orthogonal Matrices

Example 1: 2D rotation matrix. Consider the matrix

$$R_\theta = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

A direct computation shows:

$$R_\theta^\top R_\theta = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} = \begin{bmatrix} \cos^2 \theta + \sin^2 \theta & 0 \\ 0 & \cos^2 \theta + \sin^2 \theta \end{bmatrix} = I_2,$$

since $\cos^2 \theta + \sin^2 \theta = 1$. Thus R_θ is orthogonal (it represents a rotation in the plane).

Example 2: Permutation matrix. Consider, for instance,

$$P = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Its columns are the standard basis vectors e_2 and e_1 , which are orthonormal. Therefore,

$$P^\top P = I_2.$$

In fact, any matrix obtained by permuting the columns of the identity matrix is orthogonal: permutation matrices simply reorder orthonormal basis vectors.

2 Projections with Orthonormal Bases

A central reason orthonormal bases are so powerful in numerical linear algebra is that they make projections extremely simple. If the columns of a matrix A are orthonormal, then

$$A^\top A = I,$$

and the geometry of the subspace spanned by the columns becomes easy to compute.

2.1 Projection onto a Subspace

Let S be an n -dimensional subspace of $\mathbb{R}^m$ and let $q_1, \dots, q_n$ be an orthonormal basis of S .

We form the matrix

$$Q = [q_1 \ \cdots \ q_n] \in \mathbb{R}^{m \times n}.$$

By orthonormality of its columns,

$$Q^\top Q = I_n.$$

Thus Q stores the basis of S in a compact and algebraically convenient way.

2.1.1 Deriving the coefficients of the projection

Let $b \in \mathbb{R}^m$. Its orthogonal projection $\hat{b}$ onto the subspace S is the vector in S closest to b :

$$\hat{b} = \operatorname{argmin}_{y \in S} \|b - y\|.$$

Since every vector in S can be written as Qx for some $x \in \mathbb{R}^n$, the minimization problem becomes

$$\min_{x \in \mathbb{R}^n} \|b - Qx\|.$$

The normal equations for this least-squares problem are

$$Q^\top Qx = Q^\top b.$$

Using $Q^\top Q = I_n$, we immediately obtain:

$$x = Q^\top b.$$

This vector x gives the coordinates of the projection in the orthonormal basis $q_1, \dots, q_n$.

Thus the projection itself is:

$$\hat{b} = Q(Q^\top b).$$

Equivalently,

$$\boxed{\hat{b} = QQ^\top b.}$$

3 QR Decomposition: Structure and Properties

Before applying QR to least-squares problems, we first understand the factorization itself and the roles played by its two components.

3.1 Definition of the QR Decomposition

Let $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ and with linearly independent columns. Then A admits a **QR decomposition**:

$$A = QR,$$

where

- $Q \in \mathbb{R}^{m \times n}$ has **orthonormal columns**, i.e. $Q^\top Q = I_n$;
- $R \in \mathbb{R}^{n \times n}$ is square **upper triangular** and **invertible**.

The decomposition is not unique, but is unique if we additionally impose that the diagonal entries of R are positive.

Note

The columns of Q form an orthonormal basis of the column space of A :

$$\text{col}(A) = \text{col}(Q).$$

So Q captures the geometry of the subspace, while R contains the coordinates of the original columns of A in that orthonormal basis.

3.2 How QR is constructed

To obtain a QR factorization of a matrix

$$A = [a_1 \ \cdots \ a_n] \in \mathbb{R}^{m \times n},$$

we first construct a matrix Q whose columns $q_1, \dots, q_n$ form an orthonormal basis of the column space of A :

$$Q^\top Q = I_n.$$

This orthonormal basis can be obtained in many ways (Gram–Schmidt, Householder reflections, Givens rotations). The specific method does not matter; what matters is that Q satisfies

$$\text{col}(Q) = \text{col}(A).$$

3.2.1 Defining the R matrix

Once such a Q is available, we *define* the matrix

$$R := Q^\top A.$$

This definition is natural: R records how each column a_k of A decomposes in the orthonormal basis $q_1, \dots, q_n$.

Indeed, for each k :

$$Re_k = Q^\top Ae_k = Q^\top a_k,$$

so the k -th column of R is the vector of coefficients of a_k in the basis $\{q_1, \dots, q_n\}$.

Thus:

$$a_k = Q(Q^\top a_k),$$

and in matrix form:

$$A = Q(Q^\top A) = QR.$$

So the factorization $A = QR$ follows immediately from the definition $R = Q^\top A$.

3.2.2 Why $A = Q(Q^\top A)$

Write the columns of A as $A = [a_1 \cdots a_n]$ and let $Q = [q_1 \cdots q_n]$ be an orthonormal basis of the column space of A , so that $Q^\top Q = I$ and

$$\text{col}(Q) = \text{col}(A).$$

Since the q_i 's form a basis of $\text{col}(A)$, every column a_k admits a unique expansion

$$a_k = c_{1k}q_1 + \cdots + c_{nk}q_n.$$

Because the basis is orthonormal, the coefficients are

$$c_{ik} = q_i^\top a_k.$$

Thus,

$$a_k = \sum_{i=1}^n (q_i^\top a_k) q_i = Q(Q^\top a_k).$$

Stacking this identity for all columns a_k yields

$$A = Q(Q^\top A).$$

In other words, $Q^\top A$ contains the coordinates of the columns of A in the orthonormal basis given by Q , and multiplying by Q reconstructs A from those coordinates.

3.2.3 Why R is upper triangular

When constructing the orthonormal vectors $q_1, \dots, q_n$, the k -th vector q_k is built to be orthogonal to all previously constructed vectors:

$$q_k \perp q_1, \dots, q_{k-1}.$$

As a consequence, the column a_k of A can only have components along $q_1, \dots, q_k$:

$$a_k \in \text{span}\{q_1, \dots, q_k\}.$$

Therefore,

$$q_i^\top a_k = 0 \quad \text{for all } i > k.$$

Since R is defined as

$$R = Q^\top A,$$

its entries are

$$r_{ik} = q_i^\top a_k.$$

Thus,

$$r_{ik} = 0 \quad \text{whenever } i > k,$$

which means R has the structure

$$R = \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ 0 & r_{22} & \cdots & r_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & r_{nn} \end{bmatrix}.$$

Note

The triangular shape of R reflects the construction order: the k -th orthonormal direction q_k influences columns $a_k, a_{k+1}, \dots, a_n$, but not earlier ones. Each step adds a new direction orthogonal to all previous ones, so later columns cannot have a component in directions that have not yet been created.

3.3 Least Squares via QR

To solve the least-squares problem

$$\min_x \|Ax - b\|,$$

we use $A = QR$ with $Q^\top Q = I$.

Starting from the normal equations:

$$A^\top A \hat{x} = A^\top b,$$

substitute $A = QR$:

$$R^\top Q^\top QR \hat{x} = R^\top Q^\top b.$$

Since $Q^\top Q = I$,

$$R^\top R \hat{x} = R^\top Q^\top b.$$

Because R has full rank (its diagonal entries are nonzero), $R^\top$ is invertible, so we left-multiply by $(R^\top)^{-1}$:

$$R\hat{x} = Q^\top b.$$

Thus the least-squares solution is obtained by solving the triangular system

$$\boxed{R\hat{x} = Q^\top b}.$$

Note

The entire computational cost reduces to one projection $Q^\top b$ and one triangular solve. This method is significantly more stable than solving the normal equations directly.

4 Economical vs. Full QR Decomposition

When $A \in \mathbb{R}^{m \times n}$ has full column rank ($m \geq n$), there are two standard forms of the QR factorization.

4.1 Economical (thin) QR decomposition

The economical QR factorization writes

$$A = QR,$$

where

$$Q \in \mathbb{R}^{m \times n}, \quad Q^\top Q = I_n, \quad R \in \mathbb{R}^{n \times n} \text{ upper triangular and invertible.}$$

We define

$$R := Q^\top A.$$

This is the minimal QR representation: only the n orthonormal directions relevant to $\text{col}(A)$ are kept.

$$A = QR \Leftrightarrow \begin{bmatrix} A \end{bmatrix} = \begin{bmatrix} Q \end{bmatrix} \begin{bmatrix} R \end{bmatrix}.$$

Figure 2: Economical (thin) QR decomposition $A = QR$. Image from the NumCS(E) script by Prof. Ralf Hiptmair.

Note

This version is used in almost all numerical linear algebra routines where only the column space of A matters.

4.2 Full QR decomposition

To obtain a full QR decomposition, we extend the n orthonormal columns of Q to an orthonormal basis of $\mathbb{R}^m$:

$$\tilde{Q} = [q_1 \cdots q_n \mid q_{n+1} \cdots q_m] \in \mathbb{R}^{m \times m}, \quad \tilde{Q}^\top \tilde{Q} = I_m.$$

Then

$$A = \tilde{Q} \begin{bmatrix} R \\ 0 \end{bmatrix},$$

where the lower block is an $(m - n) \times n$ zero matrix.

Equivalently,

$$\tilde{Q}^\top A = \begin{bmatrix} R \\ 0 \end{bmatrix}.$$

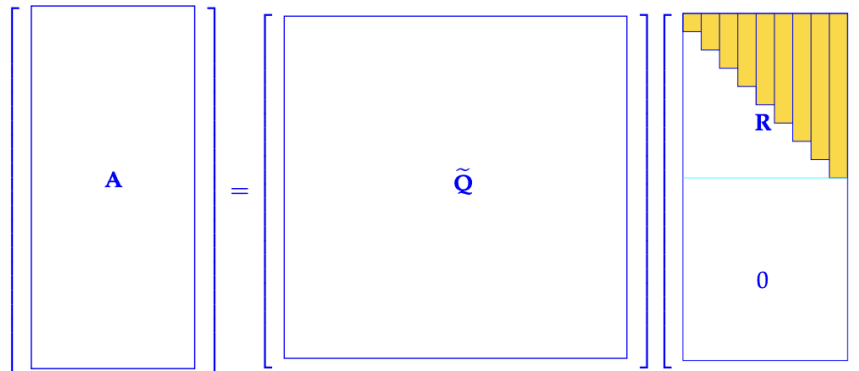


Figure 3: Full QR decomposition $A = \tilde{Q} \begin{bmatrix} R \\ 0 \end{bmatrix}$. Image adapted from the NumCS(E) script by Prof. Ralf Hiptmair.

Note

The full QR factorization is useful when a complete orthogonal change of coordinates in $\mathbb{R}^m$ is required. Householder QR and Givens QR compute this version internally.

4.3 Comparison

$A = QR$ (economical, thin)
$A = \tilde{Q} \begin{bmatrix} R \\ 0 \end{bmatrix}$ (full orthogonal basis of $\mathbb{R}^m$)

- The economical QR is smaller and computationally cheaper.
- The full QR includes the orthogonal complement of $\text{col}(A)$.
- Inside numerical libraries, the thin QR is almost always computed.

5 Why Classical Gram–Schmidt Cannot Be Used in Practice

Up to this point, we have seen how useful the QR decomposition is:

$$A = QR, \quad R = Q^\top A,$$

and how it naturally appears in projections, least-squares problems, and orthogonal coordinate transformations.

A natural question now arises:

How do we actually compute Q and R from a given matrix A ?

The most familiar procedure from linear algebra is the Classical Gram–Schmidt algorithm. It is conceptually simple and produces orthonormal vectors directly. However, despite being mathematically correct, it is *not usable* on a real computer.

Why? Because floating-point arithmetic severely amplifies the numerical instabilities inherent in the algorithm.

5.1 Geometric intuition for the instability

Consider two vectors a and b in $\mathbb{R}^m$ that are almost parallel. Classical Gram–Schmidt forms

$$g = b - (b^\top q_1)q_1,$$

where $q_1 = a/\|a\|$.

If a and b point in nearly the same direction, then

$$(b^\top q_1)q_1 \approx b,$$

and the subtraction produces

$$g \approx b - b,$$

that is, a vector with *extremely small length*. In floating-point arithmetic, this means that g is almost swallowed by roundoff error, and the computed $q_2 = g/\|g\|$ is essentially meaningless.

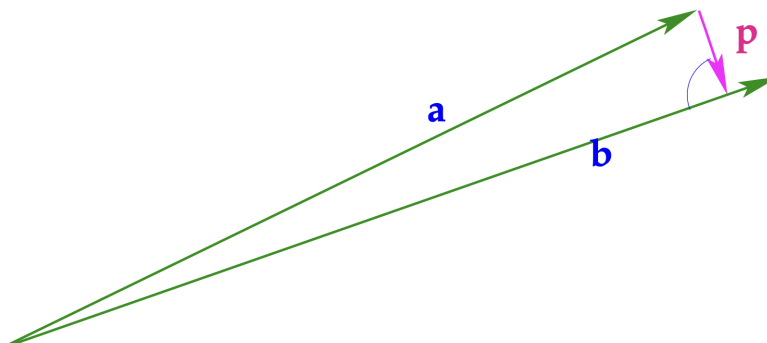


Figure 4: Two almost parallel vectors a and b , illustrating why Classical Gram–Schmidt becomes numerically unstable: the residual vector p is extremely small. Figure adapted from the NumCS(E) script by Prof. Ralf Hiptmair.

Geometrically, the algorithm tries to build a perpendicular direction out of a subtraction of two nearly identical vectors. This operation is one of the most unstable ones that a computer can perform.

5.2 Consequences

As a result, Classical Gram–Schmidt

- fails to produce truly orthonormal vectors,
- accumulates floating–point error exponentially,
- and is completely unsuitable for computing the QR decomposition.

Note

In practice, Gram–Schmidt is used only as a theoretical tool. Real numerical software never uses it to compute QR.

5.3 What is used instead?

Since Gram–Schmidt is unreliable, numerical linear algebra relies on *orthogonal transformations* that are stable by construction: reflections and rotations.

These lead to the two industry-standard algorithms:

- **Householder QR** (reflection-based),
- **Givens QR** (rotation-based).

We introduce them next.

6 Computation of Orthogonal Transformations

We now turn to the central practical question:

How can we actually construct an orthogonal matrix in order to transform a given vector or a given matrix in a controlled way?

The first key observation is the following.

6.1 A structural fact

Corollary. The product of two orthogonal matrices of the same size is again orthogonal.

$$Q_1, Q_2 \in \mathbb{R}^{n \times n} \text{ orthogonal} \implies Q_1 Q_2 \text{ orthogonal.}$$

This means that all orthogonal matrices together form a group under matrix multiplication. In particular, *any orthogonal transformation can be built as a product of simpler orthogonal transformations.*

This is the main idea behind all practical QR algorithms.

Note

Although these orthogonal transformations live in $\mathbb{R}^{n \times n}$ and therefore describe *full* n -dimensional operations, this conceptual viewpoint is crucial. Even though real numerical libraries (such as those behind Python, MATLAB or NumPy) never form these full matrices explicitly, the theory works at the level of $n \times n$ orthogonal transformations because it makes all arguments clear and transparent.

6.2 Reflections

6.3 Why reflections are useful

Consider a vector $a \in \mathbb{R}^2$ that forms some angle with the x_1 -axis. A reflection across the bisector of this angle maps a *exactly* onto the axis:

$$a \mapsto a'.$$

Geometrically, the bisector is the line that is equidistant from both a and the x_1 -axis. Reflecting a across this line produces a new vector a' lying on the axis, as illustrated in the figure.

The essential point is this:

A reflection can rotate a vector into a coordinate direction without changing its length. It can be implemented using an orthogonal matrix

This is exactly the building block we need. If we can perform this operation on each column vector of a matrix, step by step, then we can transform any set of linearly independent vectors into the standard basis vectors $e_1, e_2, \dots, e_n$.

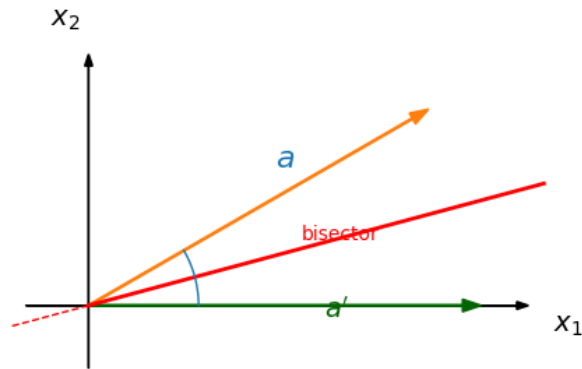


Figure 5: Reflection of a vector a across the bisector line, producing a new vector a' lying on the x_1 -axis.

And since the standard basis is orthonormal, we obtain an orthonormal basis starting from arbitrary input vectors.

Note

The standard basis $\{e_1, \dots, e_n\}$ is the simplest orthonormal basis in $\mathbb{R}^n$. By successively reflecting each vector until it aligns with one of the coordinate axes, we systematically construct an orthonormal basis using only orthogonal transformations.

6.3.1 Householder Reflections

Householder reflections are the higher-dimensional analogue of reflecting a vector across the bisector of two directions.

Given nonzero vectors $a, b \in \mathbb{R}^n$ with $\|a\| = \|b\|$, we wish to construct an orthogonal matrix H such that

$$Ha = b.$$

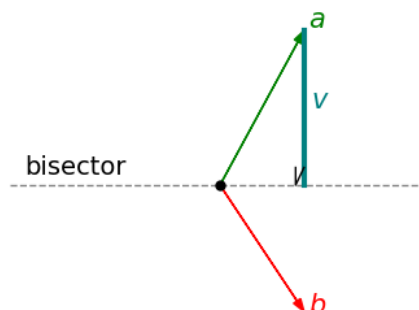


Figure 6: Geometric interpretation of a Householder reflection: the vectors a and b are symmetric with respect to the bisector (dashed line), and v is orthogonal to this bisector.

Set

$$v := a - b.$$

Notice that v is perpendicular to $a + b$ (the bisector direction). Thus reflecting a across the hyperplane orthogonal to v produces b .

The reflection of any vector x about that hyperplane is (see LinAlg)

$$x - 2 \frac{vv^\top}{v^\top v} x.$$

Applying this to $x = a$ gives

$$b = a - 2 \frac{vv^\top}{v^\top v} a = \left(I - 2 \frac{vv^\top}{v^\top v} \right) a.$$

Hence the **Householder matrix**

$$H(v) = I - 2 \frac{vv^\top}{v^\top v}$$

is orthogonal and symmetric, and satisfies $H(v)a = b$.

A very useful special case is choosing

$$v = a \pm \|a\|e_1,$$

which avoids cancellation and ensures

$$H(v)a = \pm \|a\|e_1.$$

This operation is the core of the Householder QR algorithm.

Note

Why is $H(v)$ symmetric and orthogonal? *Symmetric:* I is symmetric and $vv^\top$ is symmetric, hence

$$H(v)^\top = \left(I - 2 \frac{vv^\top}{v^\top v} \right)^\top = I - 2 \frac{(vv^\top)^\top}{v^\top v} = I - 2 \frac{vv^\top}{v^\top v} = H(v).$$

Orthogonal: We use $H(v)^\top = H(v)$ and compute

$$H(v)^2 = \left(I - 2 \frac{vv^\top}{v^\top v} \right)^2 = I - 4 \frac{vv^\top}{v^\top v} + 4 \frac{vv^\top vv^\top}{(v^\top v)^2}.$$

Since $vv^\top v = (v^\top v)v$, we get $vv^\top vv^\top = (v^\top v)vv^\top$, so the last two terms cancel: $H(v)^2 = I$. Hence $H(v)^\top H(v) = I$ and $H(v)$ is orthogonal.

6.4 Householder QR algorithm

We now use Householder reflections to compute the QR factorization of

$$A \in \mathbb{R}^{m \times n}, \quad m \geq n.$$

Idea (picture). For the first column of A we build a Householder reflector H_1 such that

$$H_1 a_1 = \pm \|a_1\| e_1.$$

If we apply H_1 to the *whole* matrix A (not only to a_1), the entire first column becomes zero below the first entry, while the remaining columns are just rotated:

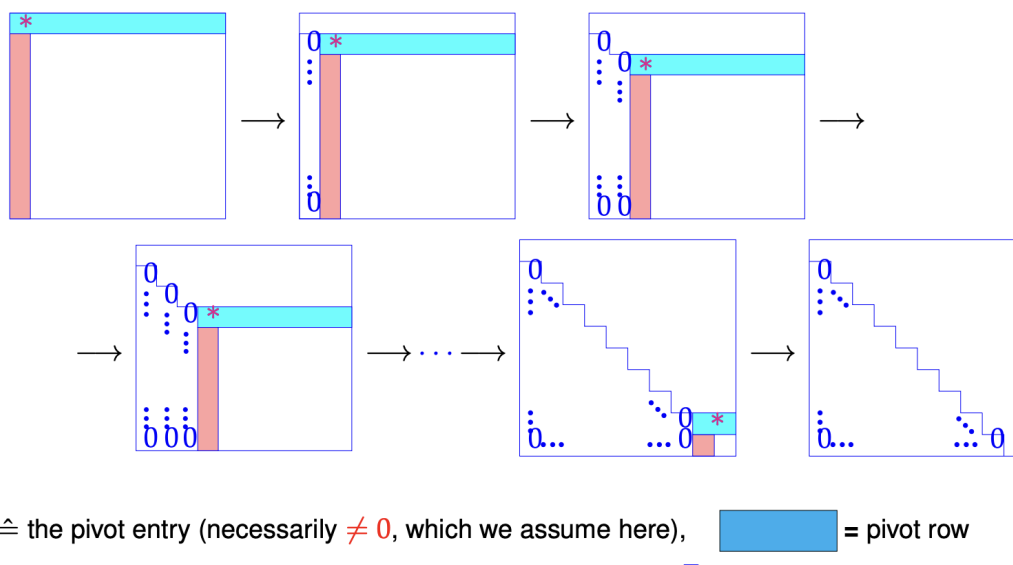


Figure adapted from the NumCS(E) script by Prof. Ralf Hiptmair.

We then repeat the same construction on the trailing submatrix, using a new reflector that acts only on the remaining rows.

Step-by-step construction

Let $A^{(1)} := A$.

1. **First column.** Take the first column of $A^{(1)}$:

$$x = A_{:,1}^{(1)} \in \mathbb{R}^m.$$

Choose

$$v_1 = x \pm \|x\| e_1,$$

and define the Householder reflector

$$H_1 = I_m - 2 \frac{v_1 v_1^\top}{v_1^\top v_1}.$$

Then

$$A^{(2)} := H_1 A^{(1)}$$

has the form

$$A^{(2)} = \begin{bmatrix} * & * & \cdots & * \\ 0 & & & \\ \vdots & \tilde{A}^{(2)} & & \\ 0 & & & \end{bmatrix},$$

i.e. the first column is zero below the diagonal.

2. **Second column.** Now look at the submatrix $\tilde{A}^{(2)}$ obtained by deleting the first row and first column. Take its first column:

$$x = \tilde{A}_{:,1}^{(2)} \in \mathbb{R}^{m-1}.$$

Build a reflector $H_2^{(\text{small})}$ in $\mathbb{R}^{m-1}$ as before, and embed it into an $m \times m$ matrix:

$$\tilde{H}_2 = \begin{bmatrix} 1 & 0 \\ 0 & H_2^{(\text{small})} \end{bmatrix}.$$

Then

$$A^{(3)} := \tilde{H}_2 A^{(2)}$$

has zeros below the diagonal also in the second column.

3. **Continue.** We continue this process for $k = 1, \dots, n$, each time constructing a Householder reflector that acts only on the lower $(m - k + 1)$ rows and zeroes out the entries below the diagonal in column k .

After n steps we obtain an upper triangular matrix:

$$A^{(n+1)} = R,$$

and

$$A^{(n+1)} = \tilde{H}_n \cdots \tilde{H}_2 H_1 A \implies A = QR,$$

where

$$Q = H_1^\top \tilde{H}_2^\top \cdots \tilde{H}_n^\top = H_1 \tilde{H}_2 \cdots \tilde{H}_n$$

is orthogonal (a product of orthogonal matrices).

Note

Each Householder step acts like a “mirror” that straightens one column of A into a coordinate direction and kills all entries below the diagonal. Because we only use orthogonal transformations, the algorithm is numerically stable and produces the QR factorization $A = QR$ with Q orthogonal and R upper triangular.

6.5 Rotations (Givens Rotations)

Rotations are the second fundamental building block for constructing orthogonal transformations. Unlike Householder reflectors, which act on an entire vector at once, **Givens rotations modify**

only two entries at a time, leaving the rest unchanged.

2D Rotation. In $\mathbb{R}^2$, a rotation by an angle θ is represented by

$$G(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}.$$

This matrix is orthogonal, preserves lengths, and rotates any vector by θ .

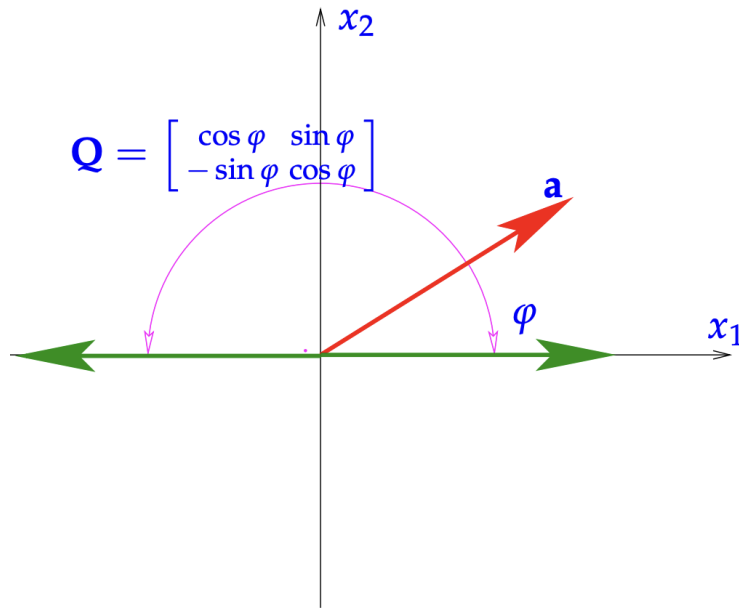


Figure 7: Rotation of a vector in the (x_1, x_2) -plane by an angle φ . Figure adapted from the *Numerical Methods for CSE* script by Prof. Ralf Hiptmair.

Givens rotations in higher dimensions

To eliminate an entry a_k of a vector $a \in \mathbb{R}^n$, we apply a Givens rotation in the $(1, k)$ -plane. The matrix $G_{1k}(c, s)$ is the identity except for a 2×2 block in rows/columns 1 and k :

$$G_{1k}(c, s) = \begin{bmatrix} c & & \cdots & s \\ & 1 & & \\ & & \ddots & \\ \vdots & & & \ddots & \\ -s & & & & c \\ & & \cdots & & & I \end{bmatrix},$$

where the nontrivial entries are

$$(G_{1k})_{11} = c, \quad (G_{1k})_{1k} = s, \quad (G_{1k})_{k1} = -s, \quad (G_{1k})_{kk} = c,$$

and all other diagonal entries are 1.

Applying $G_{1k}(c, s)$ to a changes only the components a_1 and a_k :

$$G_{1k}(c, s) \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_k \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} ca_1 + sa_k \\ a_2 \\ \vdots \\ -sa_1 + ca_k \\ \vdots \\ a_n \end{bmatrix}.$$

To annihilate a_k we choose c, s such that

$$-sa_1 + ca_k = 0 \implies c = \frac{a_1}{\sqrt{a_1^2 + a_k^2}}, \quad s = \frac{a_k}{\sqrt{a_1^2 + a_k^2}}.$$

Note

The denominator $\sqrt{a_1^2 + a_k^2}$ is essential: it ensures that the parameters c and s satisfy

$$c^2 + s^2 = 1,$$

which is the defining condition for the 2×2 rotation block to be orthogonal.

Indeed, with

$$c = \frac{a_1}{\sqrt{a_1^2 + a_k^2}}, \quad s = \frac{a_k}{\sqrt{a_1^2 + a_k^2}},$$

we have

$$c^2 + s^2 = \frac{a_1^2}{a_1^2 + a_k^2} + \frac{a_k^2}{a_1^2 + a_k^2} = 1.$$

Therefore the 2×2 block

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

is orthogonal, and so the full Givens matrix $G_{1k}(c, s)$ is orthogonal as well:

$$G_{1k}^\top G_{1k} = I.$$

This is precisely why we normalize by $\sqrt{a_1^2 + a_k^2}$ — without this normalization, the transformation would not preserve lengths and would fail to be orthogonal (or, in other words, a rotation matrix).

Applying rotations to a full matrix

To eliminate the subdiagonal entries of the first column of A , we apply rotations one at a time:

$$a^{(1)} \xrightarrow{G_{12}} a^{(2)} \xrightarrow{G_{13}} a^{(3)} \dots \xrightarrow{G_{1n}} \begin{bmatrix} \pm \|a\| \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Note

Each Givens rotation $G_{1k}(c, s)$ eliminates one component a_k of the vector while preserving the Euclidean norm, because G_{1k} is orthogonal. Thus, after applying the sequence of rotations

$$G_{12}, G_{13}, \dots, G_{1n},$$

the resulting vector is of the form

$$\begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad \text{with } \alpha^2 = a_1^2 + a_2^2 + \dots + a_n^2.$$

Since orthogonal transformations preserve lengths,

$$\left\| \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix} \right\| = \|a\|,$$

and therefore $\alpha = \|a\|$ (up to a sign, which can always be made positive by choosing the sign convention for c and s). Hence the sequence of rotations maps a to

$$\begin{bmatrix} \pm \|a\| \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Applying the same rotations to all columns yields

$$G_{1n} \dots G_{13} G_{12} A = \begin{bmatrix} * & * & \dots \\ 0 & * & \dots \\ \vdots & & \ddots \end{bmatrix}.$$

Repeating the procedure on the remaining submatrix produces R after

$$(m-1) + (m-2) + \cdots + (m-n) = \frac{n(m-1)}{2}$$

rotations.

Thus the full factorization is

$$A = Q^\top R, \quad Q = G_{12}^\top G_{13}^\top \cdots G_{mn}^\top.$$

Note

Givens rotations are ideal when A is sparse or when only a few entries need to be eliminated—they touch only two rows at a time.

Efficient storage of Givens rotations

A single rotation

$$G = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

can be stored using only one number:

$$\rho = \begin{cases} 1, & s = 0, \\ \frac{s}{2} \operatorname{sgn}(c), & |s| < |c|, \\ 2 \operatorname{sgn}(s)/c, & |c| \leq |s|. \end{cases}$$

From ρ we can always reconstruct c and s without cancellation errors:

$$\begin{aligned} |\rho| < 1 &\Rightarrow s = 2\rho, \quad c = \sqrt{1 - s^2} \\ |\rho| > 1 &\Rightarrow c = \frac{2}{\rho}, \quad s = \sqrt{1 - c^2}. \end{aligned}$$

Note

This avoids catastrophic cancellation and is the classical way Givens rotations are stored in numerical linear algebra libraries (LAPACK, BLAS).

7 Computational cost of Givens QR - Optional

To eliminate the subdiagonal entries of column k of $A \in \mathbb{R}^{m \times n}$, we apply one Givens rotation per entry:

$$(m - k) \text{ rotations for column } k.$$

Summing over all $k = 1, \dots, n$ gives a total of

$$N_{\text{rot}} = \sum_{k=1}^n (m - k) = mn - \frac{n(n+1)}{2}.$$

Cost of applying a single rotation. A Givens rotation $G_{ik}(c, s)$ acts on two rows of A : rows i and k . However, these two rows contain all remaining columns

$$k, k + 1, \dots, n.$$

Hence one rotation requires updating roughly

$$(\# \text{ of active columns}) \approx n - k + 1$$

entries *twice* (once for each row). Each update requires about 6 floating point operations:

$$\begin{aligned}\tilde{a}_i &= c a_i + s a_k, \\ \tilde{a}_k &= -s a_i + c a_k.\end{aligned}$$

Thus a single rotation costs approximately

$$\text{flops}(k) \approx 6(n - k + 1).$$

Total cost (dense case). Summing over all rotations needed to eliminate column k :

$$\text{flops}_k \approx \sum_{j=k+1}^m 6(n - k + 1) = 6(m - k)(n - k + 1).$$

Finally,

$$\text{flops}_{\text{total}} \approx \sum_{k=1}^n 6(m - k)(n - k + 1) = 3mn^2 - n^3 + \mathcal{O}(mn).$$

Thus Givens QR costs on the order of

$$\boxed{\mathcal{O}(mn^2)}.$$

Note

Even though each Givens rotation modifies only two rows, each rotation must be applied to the *entire remaining part of the matrix* (all active columns). This makes the method significantly more expensive than Householder QR for dense matrices, where Householder reflections eliminate all subdiagonal entries of a column *in a single step*. However, Givens rotations become superior when A is sparse or when only a small number of entries need to be modified (for example, in updating an existing QR factorization). In such cases, touching only two rows at a time is a major advantage.

8 Computational cost of Householder QR - Optional

We now estimate the cost of computing the QR factorization of $A \in \mathbb{R}^{m \times n}$ ($m \geq n$) using Householder reflections.

At step k , we construct a reflector that acts only on the *trailing* submatrix of size $(m - k + 1) \times (n - k + 1)$ and eliminates all entries below the diagonal in column k in *one shot*.

Step k : building the reflector. At step k , we consider the vector

$$x = A_{k:m,k}^{(k)} \in \mathbb{R}^{m-k+1},$$

i.e. the part of column k from row k downwards.

We form

$$v = x \pm \|x\|e_1,$$

normalize it if desired, and then apply the formula

$$H_k = I - 2 \frac{vv^\top}{v^\top v}.$$

Cost to build v and the scalars (norm, sign, etc.) is on the order of

$$\mathcal{O}(m - k + 1) \text{ flops},$$

which is negligible compared to applying H_k to the whole $(m - k + 1) \times (n - k + 1)$ submatrix.

Step k : applying the reflector. Instead of forming H_k explicitly, we use the rank-one structure:

$$H_k A^{(k)} = A^{(k)} - 2v \left(\frac{v^\top A^{(k)}}{v^\top v} \right).$$

This consists of:

- computing $w^\top = v^\top A^{(k)}$ (one vector–matrix product over the trailing submatrix),
- updating the trailing submatrix via $A^{(k+1)} = A^{(k)} - 2vw^\top / (v^\top v)$.

Here $A^{(k)}$ (restricted to rows $k : m$ and columns $k : n$) has size $(m - k + 1) \times (n - k + 1)$.

- Computing $w^\top = v^\top A^{(k)}$ costs roughly

$$2(m - k + 1)(n - k + 1) \text{ flops},$$

(one multiply + one add per entry). - Updating the submatrix $A^{(k+1)} = A^{(k)} - 2vw^\top / (v^\top v)$ costs again about

$$2(m - k + 1)(n - k + 1) \text{ flops}.$$

So the dominant cost of step k is

$$\text{flops}_k \approx 4(m - k + 1)(n - k + 1).$$

Total cost. Summing over $k = 1, \dots, n$:

$$\text{flops}_{\text{total}} \approx \sum_{k=1}^n 4(m - k + 1)(n - k + 1).$$

For $m \geq n$ and a dense matrix, this simplifies to the classical estimate

$$\text{flops}_{\text{total}} \approx 2mn^2 - \frac{2}{3}n^3 \quad (\text{leading terms}).$$

Thus Householder QR also costs $\mathcal{O}(mn^2)$, but with a smaller constant than Givens QR.

Note

For dense matrices, Householder QR is the method of choice:

- each reflector annihilates an entire column below the diagonal in a single step,
- the algorithm is numerically stable (uses only orthogonal transforms),
- and the total cost $\approx 2mn^2 - \frac{2}{3}n^3$ flops is typically lower than for Givens QR.

In contrast, Givens rotations eliminate one entry at a time and must be applied to all active columns, which makes them more expensive for dense problems but very attractive for sparse matrices or rank-one updates.

9 Householder and Givens in Python

Modern numerical software already implements the orthogonal transformations discussed above. In particular, Python (NumPy and SciPy) provides direct access to QR decompositions and even to the underlying Householder and Givens operations.

9.1 Householder QR in NumPy

NumPy computes QR factorizations using Householder reflectors (LAPACK's `dgeqrf` routine):

$$A = QR.$$

The basic interface is:

```
import numpy as np
Q, R = np.linalg.qr(A)                # full QR
Q, R = np.linalg.qr(A, mode="reduced") # economical QR
```

9.2 Householder reflectors in SciPy

SciPy provides direct access to the vector that defines a Householder transformation. Given a nonzero vector x , the function

```
from scipy.linalg import householder
v = householder(x)
```

returns the *Householder direction* v used to construct the reflector

$$H = I - 2 \frac{vv^T}{v^T v}.$$

9.3 Givens rotations in SciPy

SciPy also provides explicit Givens rotations:

```
from scipy.linalg import givens
c, s = givens(a, b)
```

The returned coefficients satisfy

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sqrt{a^2 + b^2} \\ 0 \end{bmatrix},$$

making it easy to illustrate entry-by-entry elimination.

When Givens is used internally

Dense QR uses Householder reflectors by default. However, *sparse* routines (e.g. `lsqr`, `lsmr`, `spsolve`) rely heavily on Givens rotations since each rotation touches only two rows and preserves sparsity patterns.

Note

The Python ecosystem mirrors the algorithms used in high-performance numerical linear algebra libraries (BLAS/LAPACK). Householder reflectors are preferred for dense matrices due to their lower cost, while Givens rotations are ideal when sparsity must be preserved.